

Estudos P1

Sunday, 12 October 2025

14:29

EXERCÍCIOS DE ACOMPANHAMENTO 2022/1

1) Discorra sobre programação paralela e concorrente, e aplicações multi-thread, paralelas e distribuídas. Cite e comente duas das principais dificuldades na implementação de aplicações concorrentes.

P vs C: Concorrência permite que múltiplas tarefas executem intercaladamente, compartilhando recursos; paralelismo permite que múltiplas tarefas executem simultaneamente, em vários núcleos para ganhar desempenho. Concorrência é fingir pro programa que ele tem acesso a vários recursos, enquanto na verdade tu tá só escalonando, enquanto paralelismo ele tem mesmo.

MT vs P vs D:

- aplicações multi-thread: usam várias threads dentro de um processo, melhor aproveitando a CPU; boa forma de organizar sistemas de software modernos
- aplicações paralelas: processos executando fisicamente ao mesmo tempo em diferentes processadores, para resolver um problema mais rapidamente ou maior
- aplicações distribuídas: rodam em máquinas/hosts diferentes, comunicando-se por rede, para distribuir trabalho, acessar dados distribuídos e computação paralela em larga-escala sobre a rede

Principais dificuldades da programação concorrente:

- Race Condition: quando threads acessam variáveis compartilhadas sem sincronização, gerando resultados incorretos
- Deadlock: quando threads ficam bloqueadas esperando por recursos uns dos outros
- Starvation: quando uma thread nunca é escalonada por falta de acesso a recursos

2) Discorra sobre a necessidade de sincronização de processos e explique quais as propriedades que sistemas concorrentes e distribuídos devem satisfazer.

Em sistemas concorrentes, onde múltiplos processos e threads executam em paralelo e compartilham recursos, é essencial coordenar tal acesso para garantir resultados corretos. Caso contrário, diferentes processos podem interferir entre si e modificar variáveis descoordenadamente, gerando condições de corrida e inconsistência.

Para funcionarem corretamente, sistemas concorrentes e distribuídos devem satisfazer:

- exclusão mútua: apenas um processo por vez acessa uma seção crítica
- progresso: se nenhum processo estiver na seção crítica, algum que queira deve eventualmente conseguir entrar, ou seja o sistema não pode ficar parado indefinidamente
- espera limitada: um processo não pode esperar até a eternidade pra entrar na seção crítica
- ordem e consistência de eventos: em sistemas distribuídos, a sincronização depende de mensagens e eventos temporais

3) Comente as principais características das implementações baseadas em software e hardware para o operador de exclusão mútua. Em particular, quais os motivos que tornam implementações baseadas em software pouco apropriadas para uso em programação concorrente?

Exclusão mútua via software:

- sem suporte do processador, feito apenas com lógica de controle entre processos (variáveis compartilhadas, flags e turnos)
- algoritmos como de Peterson e de Dekker
- úteis para fins didáticos

Exclusão mútua via hardware:

- usam instruções atômicas do processador como `test-and-set`, `swap`, `compare-and-swap`
- executam a operação crítica de forma atômica (sem interferência de outras threads e sem interrupção)
- são a base para mutexes, semáforos, etc

Implementações baseadas em software são pouco apropriadas porque são ineficientes (fazem busy-wait e desperdiçam CPU), difíceis de escalar para múltiplos núcleos e processos, mais sujeitas a erros de sincronização devido a reordenação de instruções

de instruções.

4) Idealize (de forma breve) um sistema simples com múltiplos processos concorrentes com necessidade de controle de acesso a um recurso comum, e explique como o conceito de semáforos contadores pode ser utilizado para resolver o problema de acesso a seção crítica nesse sistema. Em particular, comente por que o uso de semáforo contador é mais conveniente no sistema idealizado em comparação ao uso de semáforo binário ou mutex.

Sistema: impressão compartilhada, com N processos e M impressoras disponíveis. Cada processo precisa acessar uma impressora para imprimir, com no máximo M processos fazendo isso simultaneamente.

Utilização de semáforos contadores: o semáforo é inicializado com M para controlar o acesso, onde cada processo executa `sem_wait()` ao entrar (ocupando uma vaga) e `sem_post()` ao sair (liberando uma vaga). Assim, a variável do semáforo reflete o número de vagas (recursos) livres, garantindo que nunca mais de M processos entrem na seção crítica (imprimir algo).

Por que semáforo contador é melhor? Usar semáforos contadores é mais conveniente do que um binário ou mutex porque estes permitem apenas um processo por vez, limitando o paralelismo desnecessariamente e desperdiçando recursos. Além disso, o semáforo contador evita busy-wait e é suficiente para o cenário onde qualquer processo pode usar qualquer impressora.

5) O que são e para que servem as primitivas wait e signal? Enumere e explique as estratégias de sinalização empregadas na implementação de monitores.

Um monitor é uma estrutura de sincronização de alto nível que encapsula variáveis compartilhadas, os métodos que as manipulam e o acesso. As primitivas wait e signal servem para sincronizar a execução entre processos ou threads que compartilham um monitor:

- wait: usada quando uma thread não pode prosseguir porque uma certa condição não é verdadeira. Ao chamar `wait()`, ela libera o monitor e entra numa fila de espera associada à condição
- signal: usada por outra thread quando ela altera o sistema de forma que a condição mencionada pode ter se tornado verdadeira. Ela acorda uma thread que estava esperando na fila

Estratégias de sinalização em monitores:

- Sinalização imediata:
 - o conhecida como signal-and-continue ou Mesa semantics
 - o a thread que chama `signal()` continua executando dentro do monitor, e a thread sinalizada fica esperando até a liberação
 - o é simples e eficiente, mas a condição pode ter mudado até a thread acordar (por isso o `while(condition) wait()`)
- Sinalização direta:
 - o conhecida como signal-and-wait ou Hoare semantics
 - o a thread que chama `signal()` cede imediatamente o controle do monitor pra thread sinalizada, que volta a executar imediatamente
 - o mais previsível, mas mais complexa de implementar

EXERCÍCIOS DE ACOMPANHAMENTO 2022/1

1) Considere a especificação de um sistema para troca de mensagens entre pessoas, usando uma arquitetura na qual uma aplicação servidor coordena a troca de mensagens, e várias aplicações clientes interagem com o servidor para envio e recebimento de mensagens. Cada aplicação executa em um dispositivo diferente. Para viabilizar a troca de mensagens entre as aplicações cliente e servidor, a implementação desse sistema deverá usar um ou mais mecanismos para comunicação inter-processo. Explique por que mecanismos de pipe e de memória compartilhada não podem ser usados na implementação da comunicação inter-processos nesse sistema. Para a implementação desse sistema, qual (ou quais) mecanismos de IPC seriam mais apropriados? Justifique.

Por que pipe e memória compartilhada não são adequados?

- Pipes: são canais unidirecionais usados pra transmitir bytes entre processos locais, dependendo do kernel local para manter o buffer e coordenar leitura/escrita. Não funcionam entre máquinas diferentes porque não existe um kernel compartilhado
- Memória compartilhada: permite que os processos compartilhem uma região de memória física, o que o torna o método de IPC local mais rápido. Não funciona em máquinas diferentes porque não é possível mapear uma mesma região de memória em dois computadores

mesma região de memória em dois computadores

Pela descrição do sistema, fica claro que ele é distribuído, visto que há um servidor e vários clientes rodando em dispositivos diferentes. Ou seja, pelo fato de que pipe e memória compartilhada serem mecanismos intra-máquina, eles não são adequados para nosso sistema de mensagens.

Quais mecanismos seriam mais apropriados?

- O mecanismo mais adequado e mais comum para protocolos cliente-servidor são sockets, uma vez que permitem comunicação entre processos em máquinas diferentes através da rede. Podem usar TCP ou UDP
- Outras alternativas são Remote Procedure Call ou Remote Method Invocation, que abstraem os sockets para chamadas remotas de métodos

2) Defina comunicação síncrona e assíncrona de processos. É possível implementar comunicação assíncrona sem uso de mecanismos de buferização? Por quê? Quanto à comunicação síncrona, discorra um cenário simples na qual ela poderia levar a deadlock.

Na comunicação síncrona, as primitivas send e receive são bloqueantes, ou seja, o processo emissor fica bloqueado até que a recepção seja efetivada e o processo receptor fica bloqueado até que o envio seja feito e a mensagem recebida. Na comunicação assíncrona, pelo menos uma primitiva é não-bloqueante, logo o emissor não espera o receptor: ele envia a mensagem e continua executando.

Não é possível implementar comunicação assíncrona sem um buffer, visto que o emissor precisa ter onde armazenar a mensagem enquanto o receptor ainda não a consumiu.

Um exemplo comum de deadlock em comunicação síncrona é quando dois processos enviam mensagens ao mesmo tempo, o que faz com que ambos fiquem bloqueados esperando que o outro receba a mensagem.

3) Considere a especificação de um sistema para o envio de arquivos de um computador a outro, através da internet. O sistema será formado por uma aplicação servidor, que aguarda o recebimento do arquivo, e uma aplicação cliente, que faz o envio do arquivo ao servidor. Quais as três dificuldades adicionais que o programador desse sistema enfrentará ao implementar essa aplicação usando o protocolo UDP ao invés do TCP?

Para esse tipo de sistema, é importante que os dados cheguem completos e na ordem correta e que o servidor consiga detectar perdas e retransmitir pacotes se necessário. O protocolo TCP já provê isso e mais automaticamente, enquanto que UDP não. Por isso, três dificuldades que o programador terá são:

- Ausência de confiabilidade: UDP não garante que os datagramas cheguem ao destino, e pacotes podem ser perdidos ou duplicados no caminho. Assim, seria necessário implementar um sistema de retransmissão e confirmação de recebimento
- Ausência de controle de ordem: pacotes UDP podem chegar fora de ordem, então como os arquivos precisam ser reconstruídos na sequência correta, o programador teria que incluir números de sequência nos pacotes e reordená-los no destino
- Ausência de controle de fluxo e congestionamento: UDP não ajusta automaticamente a taxa de envio conforme a capacidade da rede e do receptor, fazendo com que o cliente possa enviar dados muito rápido e sobrecarregar o servidor. Assim, o programador teria que implementar manualmente um mecanismo de controle de fluxo

4) Considere um sistema para o envio de arquivos entre dois dispositivos que se comunicam pela internet. O sistema é formado por duas aplicações A e B. A aplicação A envia o arquivo dividindo o mesmo em blocos, e enviando cada bloco para a aplicação B, usando UDP. Para a implementação desse sistema, pode-se imaginar o envio dos blocos dos arquivos entre A e B usando as semânticas at most once, at least once, e exactly once. Qual dessas semânticas é a mais apropriada para a implementação desse sistema? Cite os problemas que as demais semânticas podem trazer para a implementação / operação do sistema.

Semânticas:

- at most once: a mensagem pode não ser entregue, mas nunca é duplicada
- at least once: a mensagem é entregue, mas pode haver duplicatas
- exactly once: cada mensagem é entregue uma única vez

Sabendo o conceito de cada semântica, fica claro que a ideal é a exactly once, pois para o sistema de arquivos descrito, cada arquivo deve ser recebido uma única vez, na ordem correta e sem perdas. A semântica exactly once garante que cada bloco será recebido e processado uma única vez, mesmo em caso de retransmissão, falhas ou atrasos.

Se at most once for usada, os blocos podem ser perdidos, resultando em arquivos incompletos ou corrompidos; e se a at least once for usada, os blocos podem ser entregues mais de uma vez, causando duplicação de dados e inconsistência nos arquivos.

5) Na chamada remota de procedimentos, discorra sobre as funções do stub no programa cliente e no programa servidor. Cite e comente alguns dos problemas que podem ocorrer durante a chamada remota de procedimentos.

Chamada remota de procedimentos, ou RPC, é um mecanismo que permite que um programa chame uma função em outro processo como se fosse uma função local. RPC abstrai a comunicação de rede, o que envolve troca de mensagens entre cliente e servidor.

Os stubs são módulos que fazem a ponte entre a aplicação e a camada de comunicação:

- stub do cliente: atua como um representante local da função remota, recebendo a chamada do programa cliente, empacotando os parâmetros, enviando a mensagem, e desempacotando o resultado
- stub do servidor: atua como um intermediário no lado servidor, recebendo a mensagem enviada pelo stub do cliente, desempacotando os parâmetros, chamando a função real no servidor e empacotando e enviando de volta o resultado

Problemas:

- falhas na comunicação: perda de mensagem, tempo de resposta elevado
- semântica de falha: difícil saber se a função foi executada ou não quando há falha, fazendo com que uma retransmissão possa causar execução duplicada
- incompatibilidade de representação de dados: não necessariamente o cliente e o servidor tem as mesmas arquiteturas, o que exige uma conversão de dados
- latência: chamadas remotas são mais lentas por envolverem rede

PRIMEIRA VERIFICAÇÃO 2023/2

1. Considere os dois memes de Internet abaixo:



Os memes ilustram, de forma descontraída, características de dois protocolos para comunicação inter-processos discutidos em sala de aula. Quais são essas características? Considere, na sua explicação, a semântica de comunicação e o modelo de falhas (validade e integridade) de cada protocolo.

TCP: Oferece comunicação confiável, orientada à conexão e com controle de entrega, garantindo validade (dados chegam ao destino) e integridade (sem perdas ou duplicações). Sua semântica é exactly-once, com entrega ordenada e retransmissão automática.

UDP: representa comunicação não confiável e sem conexão, onde pacotes podem se perder, chegar fora de ordem ou duplicados. Seu modelo de falhas não garante validade (pode não entregar) nem integridade (pode haver perdas, erros de ordenamento e duplicação), caracterizando uma semântica at-most-once.

2. Dois módulos do projeto de Trabalho Prático da disciplina são o Gerenciador de notificações e o Gerenciador de perfis e de sessões. Esses módulos gerenciam uma lista de perfis participantes do serviço, incluindo a relação de seguidores de cada perfil, a lista de notificações emitidas por cada perfil, e a fila de notificações pendentes de envio aos seguidores. Neste contexto, qual problema de condição de corrida pode ocorrer no acesso à lista de perfis participantes? Como o conceito de “at-most-once” é violado neste caso, especificamente? Exemplifique com uma situação de operação do serviço do projeto da disciplina.

O acesso concorrente à lista de perfis participantes pode gerar condição de corrida, pois os módulos de notificações e de perfis acessam e modificam simultaneamente uma estrutura compartilhada sem sincronização. Isso pode causar leituras inconsistentes e duplicação de dados.

Em uma situação em que o gerenciador de notificações começa a enviar notificações aos seguidores do perfil A, e ao mesmo tempo o gerenciador de perfis atualiza a lista de seguidores, adicionando um novo seguidor, podemos ter uma

situação de condição de corrida. Devido à ausência de sincronização, o sistema pode ler a lista de seguidores em um estado inconsistente, fazendo com que um seguidor apareça duas vezes ou o mesmo evento seja reenqueueado.

Nessa situação, a semântica at-most-once, que garante que cada evento ou mensagem ocorra uma única vez, é violada, pois um seguidor pode receber duas notificações idênticas.

3. Ainda no contexto da questão acima: Para proteger a lista de perfis participantes de condições de corrida, uma alternativa é utilizar primitivas de exclusão mútua. Considerando as primitivas discutidas em sala de aula, qual você considera a primitiva mais apropriada para usar neste contexto? Justifique a sua resposta considerando a forma como a lista de perfis participantes é gerenciada pelos módulos envolvidos.

Temos dois módulos concorrentes que acessam a mesma estrutura (gerenciador de notificações e gerenciador de perfis acessando a lista de perfis). Ou seja, o problema descrito é local, ocorre dentro de um mesmo serviço, não entre máquinas diferentes. Logo, a proteção deve ser feita em nível de threads.

A primitiva mais apropriada é o mutex, pois a lista de perfis participantes é um recurso único e compartilhado entre múltiplas threads de um mesmo processo. O mutex garante que apenas uma thread por vez acesse a estrutura, eliminando condições de corrida sem necessidade de busy-wait.

O uso de semáforos, por exemplo, seria mais complexo e desnecessário, já que não há múltiplos recursos a controlar. Assim, o mutex oferece exclusão mútua simples, eficiente e adequada.

4. No trabalho prático, o protocolo utilizado para comunicação inter-processo foi o UDP. Qual a semântica de comunicação do UDP, e qual a semântica de comunicação implementada no trabalho prático? Por que adotou-se uma semântica de comunicação diferente da usada pelo UDP na aplicação do trabalho prático? Cite os três principais problemas que podem ocorrer caso a semântica do UDP fosse mantida na aplicação do trabalho prático. Considere na sua resposta as possíveis situações de falha.

UDP utiliza a semântica de comunicação at-most-once, em que mensagens podem ser perdidas ou entregues fora de ordem. No trabalho prático, foi implementada uma semântica mais confiável, como exactly-once, obtida por meio de mensagens ACK, numeração de mensagens e retransmissões.

Essa mudança foi necessária porque o sistema precisava garantir que todas as notificações e mensagens chegassem corretamente ao destino. Se a semântica original do UDP fosse mantida, poderiam ocorrer perdas de mensagens, entrega fora de ordem e inconsistência de dados.

5. O que dizem as propriedades safety e liveness de programação concorrente e de que forma elas podem ser violadas no trabalho prático da disciplina?

Safety: o sistema nunca entra em um estado incorreto, ou seja, as ações são executadas de forma consistente e obedecem às regras de exclusão mútua. Uma violação de safety no trabalho prático seria quando duas threads tentam acessar ao mesmo tempo a lista de perfis, gerando uma condição de corrida e dados inconsistentes, como quando um seguidor é adicionado enquanto outro módulo percorre a lista.

Liveness: as ações que deveriam ocorrer eventualmente acontecem, ou seja, o sistema não trava e threads progredem. Uma violação de liveness seria quando há o uso incorreto de primitivas de sincronização (ex.: mutex não liberado), podendo causar deadlock, como quando o módulo de notificações bloqueia a lista de perfis esperando confirmação de outro módulo que também tenta acessar a lista.

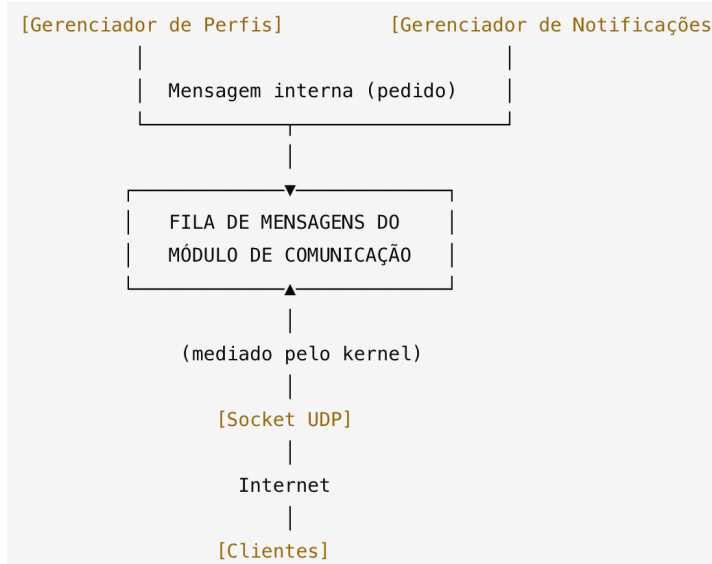
6. Considere que, além dos módulos gerenciador de notificações e gerenciador de perfis e de sessão do trabalho prático, a sua equipe implementou um módulo de comunicação. Esse módulo seria responsável por toda a troca de mensagens (comunicação interprocessos) entre o servidor e os clientes. Seguindo as boas práticas de Engenharia de Software, a implementação do módulo de comunicação da sua equipe “esconde” o socket UDP criado, oferecendo apenas uma API de mais alto nível que os subserviços de descoberta, monitoração e gerenciamento podem usar. Agora, considere que esses subserviços (cada um rodando em uma thread) precisam enviar e receber mensagens utilizando um mecanismo mediado pelo kernel do sistema operacional. Como o mecanismo de comunicação inter-processos baseado em fila de mensagens (caixa postal) pode ajudar na comunicação entre esses módulos?

O mecanismo de fila de mensagens (caixa postal) permite a comunicação assíncrona e segura entre os módulos do sistema, mediada pelo kernel. Nesse contexto, os subserviços (como notificações e perfis) podem enviar mensagens para o módulo de comunicação por meio de uma fila, sem acesso direto ao socket UDP.

Benefícios de uma fila de mensagens:

- desacoplamento entre módulos: os subserviços não precisam conhecer a implementação do socket, basta eles deixarem uma mensagem na caixa postal do módulo de comunicação. Assim, temos uma arquitetura limpa, modular e thread-safe
- comunicação assíncrona: um módulo pode enviar uma mensagem e continuar executando, pois o receptor a processará quando estiver pronto
- mediação pelo kernel: o sistema operacional garante sincronização seguro no acesso à fila, evitando condições de corrida

Diagrama mostrando o funcionamento do módulo de fila no sistema:



Um fluxo de mensagem seria da seguinte forma:

1. Cada subserviço (thread) pode enviar uma mensagem para fila do módulo de comunicação, solicitando, por exemplo, o envio de uma notificação
2. O módulo de comunicação lê as mensagens da fila, interpreta o conteúdo e envia pela rede usando o socket UDP interno
3. Quando o socket recebe algo, o módulo pode colocar as mensagens recebidas em outra fila, para que os outros módulos as processem quando estiverem prontos

7. Considere o meme de Internet abaixo:

Multithreaded programming



Fonte: CSAIL – MIT (reprodução Facebook)

O meme ilustra, de forma descontraída, desafios relacionados à programação concorrente e paralela. Nesse caso, o que representam os filhotes, os pratos e a alimentação mostrados na foto, no contexto de programação concorrente e paralela? A foto da direita ilustra qual tipo de problema na programação concorrente e paralela? Explique. Qual mecanismo pode ser usado para resolver esse problema? Explique, considerando as duas situações possíveis na sincronização entre tarefas.

Filhotes: threads ou processos executando concorrentemente

Pratos: recursos compartilhados (variáveis, buffers, estruturas de dados, memória)

Alimentação: acesso e manipulação dos dados/recursos

A foto da direita ilustra uma condição de corrida e problemas de falta de sincronização. Quando múltiplas threads acessam e modificam recursos compartilhados simultaneamente sem sincronização adequada, pode ocorrer condição de corrida (o resultado depende da ordem de execução das threads) e inconsistência de dados (threads leem/escrevem ao mesmo tempo, corrompendo dados).

O problema pode ser resolvido com sincronização. Duas situações possíveis na sincronização entre tarefas são:

- exclusão mútua: garantir que apenas um thread por vez acesse a seção crítica, através de mutexes, semáforos binários, etc. No meme, cada cachorro esperaria sua vez para acessar seu prato; enquanto um come, outros aguardam
- sincronização condicional: atrasar a execução de uma thread até que uma condição seja satisfeita, através de semáforos contadores ou variáveis de condição. No meme, um cachorro só pode comer quando houver comida disponível no seu prato ou quando for sua vez conforme alguma ordem

8. (Bônus) Livre: Elabore uma questão sobre um conceito que não seja tema de pergunta ou resposta de quaisquer das questões acima, e responda à questão elaborada de forma detalhada e justificada. A questão deve obrigatoriamente abordar um conceito pertinente à primeira etapa da disciplina. Questões ou respostas demasiadamente triviais não serão aceitas para os fins da pontuação bônus.

Explique a diferença entre concorrência e paralelismo, destacando como cada conceito é implementado em sistemas computacionais e um exemplo prático em que existe concorrência sem paralelismo.

Concorrência: a capacidade de um sistema lidar com múltiplas tarefas lógicas ao mesmo tempo. Ao invés de executá-las fisicamente em paralelo, o sistema alterna entre elas através de escalonamento de forma que pareça simultâneo.

Paralelismo: ocorre quando duas ou mais tarefas são executadas fisicamente ao mesmo tempo, utilizando múltiplos núcleos. É uma característica de hardware que permite ganho real de desempenho.

Um exemplo prático de concorrência sem paralelismo é um sistema operacional rodando em um processador single-core com múltiplos processos prontos para execução. O escalonador alterna entre eles criando a ilusão de simultaneidade: por exemplo, primeiro uma página web é carregada, depois um arquivo de texto é carregado, depois a página web é atualizada, depois o arquivo é alterado e assim por diante. Apesar de parecer que ambos acontecem ao mesmo tempo fisicamente, apenas um instrução é executado por vez.

PRIMEIRA VERIFICAÇÃO 2022/1

2) Cite as principais características de semáforos que os diferem da primitiva mutex e comente os três possíveis casos de uso de semáforos discutidos em sala.

Mutex: é um mecanismo binário (recurso está bloqueado ou liberado) usado exclusivamente para exclusão mútua. Apenas o processo que adquiriu o mutex pode liberá-lo.

Semáforos: são uma estrutura de dados com contador inteiro não negativo, ou seja, pode gerenciar a disponibilidade de vários recursos. Para isso, possui duas operações atômicas:

- P ou wait: decrementa o contador e bloqueia se for zero, indicando que todos os recursos estão sendo utilizados
- V ou signal: incrementa o contador e acorda processos, indicando que um recurso foi liberado

Assim, não necessariamente o processo que faz wait é o mesmo que faz signal.

Os três possíveis casos de uso de semáforos são:

- exclusão mútua: obtido através de um semáforo inicializado em 1; um processo realiza P antes de entrar na seção crítica e V ao sair; se assemelha a um mutex
- alocação de recursos: uso de um semáforo contador, onde o valor inicial é igual ao número de recursos disponíveis; cada P consome um recurso e cada V devolve
- sincronização de condição: quando um processo necessita realizar uma operação x somente após outro realizar y; semáforo é inicializado em 0; também conhecido como sinalização, visto que o processo da operação y sinaliza o da x

3) Quais são os dois elementos que um monitor encapsula e fale sobre as estratégias de sinalização signal and continue, signal and wait, e signal and urgent wait.

Um monitor é uma estrutura de sincronização de alto nível que encapsula dois elementos:

- variáveis permanentes (estado interno): representam os dados compartilhados entre processos e threads; são privadas ao monitor e só podem ser acessadas pelos procedimentos do próprio monitor
- procedimentos (operações): implementam as operações que manipulam as variáveis permanentes; no máximo um procedimento do monitor pode estar ativo por vez

Quando um processo executa sinaliza, é necessário definir quem vai continuar executando: quem sinalizou ou quem foi acordado. Para isso, existem três estratégias:

- signal and continue: o processo que sinalizou continua executando e o sinalizado entra para a fila de entrada do monitor
- signal and wait: o processo que sinalizou para de executar e o sinalizado assume imediatamente a execução no monitor
- signal and urgent wait: similar ao signal and wait, mas o processo que sinalizou vai para a frente da fila de entrada do monitor, sendo o próximo a executar depois do processo que foi sinalizado

4) O que é e quando ocorre uma “condição de corrida”? A que se refere o conceito de at-most-once no contexto de referência crítica?

Condição de corrida é quando dois processos acessam dados ao mesmo tempo sem sincronização, fazendo com que o resultado final das operações dependa da ordem de execução dos processos. Isso pode corromper e gerar inconsistência de dados porque um processo pode considerar que o dado possui um certo estado enquanto o outro processo altera tal estado ao mesmo tempo.

Referência crítica é uma variável que pode ser modificada por outro processo durante a execução de uma expressão. Nesse caso, em `x = e`, a propriedade at-most-once diz que

1. a expressão `e` possui no máximo uma referência crítica e `x` não é lido por outro processo, ou
2. a expressão `e` não possui qualquer referência crítica e `x` pode ser lido por outro processo.

Se alguma dessas propriedades for satisfeita, a execução de `x = e` parecerá atômica, mesmo sem ter sincronização. No exemplo abaixo, podemos ver isso em prática:

```
int x = 0, y = 0;
x = x + 1;
(new process) y = y + 1;
// sem referências críticas
```

```
int x = 0, y = 0;
x = y + 1;
(new process) y = y + 1;
// caso (1) para 'x=...', onde x faz referência a y mas não é lido por outro processo
// caso (2) para 'y=...', onde y não faz referências mas é lido pela linha a cima
// x pode ser 1 ou 2, mas y será sempre 1
```

```
int x = 0, y = 0;
x = y + 1;
(new process) y = x + 1;
// cada expressão possui uma referência crítica e cada processo atribui a uma variável lida pelo outro
// x pode ser 1 e 2, y pode ser 1 e 2 (inválido)
```

5) Cite e comente os dois modelos fundamentais para comunicação inter-processos.

Troca de mensagens (message passing):

- os processos se comunicam enviando e recebendo mensagens, sem compartilhar memória diretamente
- requer chamadas de sistema, intervenção de kernel
- melhor para compartilhamento de pequenas quantidades de dados
- usado em sistemas distribuídos

Memória compartilhada (shared-memory systems):

- os processos compartilham uma região de memória em comum para trocar dados
- chamadas de sistema apenas para estabelecer região, depois sem assistência do kernel
- comunicação mais rápida e conveniente
- requer mecanismos de sincronização

6) Quais as características da comunicação síncrona e comunicação assíncrona? Na sua resposta, comente sobre as características das primitivas send e receive no contexto de cada tipo de comunicação.

Na comunicação síncrona, os processos precisam estar sincronizados e ambas as primitivas send e receive são bloqueantes, ou seja, o emissor é bloqueado até que a mensagem seja enviada e lida, e o receptor é bloqueado até que a mensagem seja recebida e lida. Já na comunicação assíncrona, pelo menos uma primitiva é não-bloqueante, sendo a de send nunca bloqueante, ou seja, o emissor sempre continua executando após enviar a mensagem, que fica armazenada em um buffer até que o receptor a leia.

mente da linha de entrada do monitor, sendo o próximo a executar depois do processo que foi sinalizado

4) O que é e quando ocorre uma “condição de corrida”? A que se refere o conceito de at-most-once no contexto de referência crítica?

Condição de corrida é quando dois processos acessam dados ao mesmo tempo sem sincronização, fazendo com que o resultado final das operações dependa da ordem de execução dos processos. Isso pode corromper e gerar inconsistência de dados porque um processo pode considerar que o dado possui um certo estado enquanto o outro processo altera tal estado ao mesmo tempo.

Referência crítica é uma variável que pode ser modificada por outro processo durante a execução de uma expressão. Nesse caso, em ``x = e``, a propriedade at-most-once diz que

1. a expressão ``e`` possui no máximo uma referência crítica e ``x`` não é lido por outro processo, ou
2. a expressão ``e`` não possui qualquer referência crítica e ``x`` pode ser lido por outro processo.

Se alguma dessas propriedades for satisfeita, a execução de ``x = e`` parecerá atômica, mesmo sem ter sincronização. No exemplo abaixo, podemos ver isso em prática:

```
int x = 0, y = 0;
x = x + 1;
(new process) y = y + 1;
// sem referências críticas
```

```
int x = 0, y = 0;
x = y + 1;
(new process) y = y + 1;
// caso (1) para `x=...`, onde x faz referência a y mas não é lido por outro processo
// caso (2) para `y=...`, onde y não faz referências mas é lido pela linha a cima
// x pode ser 1 ou 2, mas y será sempre 1
```

```
int x = 0, y = 0;
x = y + 1;
(new process) y = x + 1;
// cada expressão possui uma referência crítica e cada processo atribui a uma variável lida
// pelo outro
// x pode ser 1 e 2, y pode ser 1 e 2 (inválido)
```

5) Cite e comente os dois modelos fundamentais para comunicação inter-processos.

Troca de mensagens (message passing):

- os processos se comunicam enviando e recebendo mensagens, sem compartilhar memória diretamente
- requer chamadas de sistema, intervenção de kernel
- melhor para compartilhamento de pequenas quantidades de dados
- usado em sistemas distribuídos

Memória compartilhada (shared-memory systems):

- os processos compartilham uma região de memória em comum para trocar dados
- chamadas de sistema apenas para estabelecer região, depois sem assistência do kernel
- comunicação mais rápida e conveniente
- requer mecanismos de sincronização

6) Quais as características da comunicação síncrona e comunicação assíncrona? Na sua resposta, comente sobre as características das primitivas send e receive no contexto de cada tipo de comunicação.

Na comunicação síncrona, os processos precisam estar sincronizados e ambas as primitivas send e receive são bloqueantes, ou seja, o emissor é bloqueado até que a mensagem seja enviada e lida, e o receptor é bloqueado até que a mensagem seja recebida e lida. Já na comunicação assíncrona, pelo menos uma primitiva é não-bloqueante, sendo a de send nunca bloqueante, ou seja, o emissor sempre continua executando após enviar a mensagem, que fica armazenada em um buffer até que o receptor a leia.

um buffer até que o receptor a leia.

7) Cite e comente as quatro propriedades esperadas para primitivas de exclusão mútua.

Exclusão mútua: apenas um processo deve acessar uma seção crítica por vez

Progresso: um processo fora da sessão crítica não pode impedir outro de entrar

Espera limitada: um processo não pode esperar indefinidamente para entrar na sessão crítica

Independência de velocidade relativa: a solução não deve depender da velocidade relativa dos processos ou do número de CPUs, garantindo que a exclusão mútua funcione em ambientes distribuídos

8) Considerando as discussões em sala de aula, classifique os protocolos TCP e UDP quanto à semântica utilizada: at most once, at least once e exactly once. Justifique a classificação a partir das características de cada protocolo.

TCP utiliza a semântica at-most-once, porque é um protocolo orientado à conexão, que garante entrega sem duplicação, ordem correta das mensagens e detecção e retransmissão de pacotes perdidos. Assim, cada mensagem é entregue no máximo uma vez ao processo receptor, mesmo que ocorra retransmissões internas (essas são transparentes para a aplicação).

UDP utiliza a semântica at-least-once, porque não é orientado à conexão, sem garantias de entrega, ordem e integridade (além do checksum básico). Como a aplicação pode implementar retransmissões para tentar garantir entrega, isso pode levar a duplicações, visto que UDP não verifica se um pacote é duplicado.

Nenhum dos dois garante exactly-once, visto que seria necessário garantir entrega confiável, sem duplicação e sem perda. TCP chega perto, mas não garante contra falhas extremas como crash após entrega e antes do ACK da aplicação.

SUPOSTA PRIMEIRA VERIFICAÇÃO 2025/2 - PROF. EDER JOHN

1) Discorra sobre programação paralela e concorrente, e aplicações multi-thread, paralelas e distribuídas. Cite e comente duas das principais dificuldades na implementação de aplicações concorrentes.

P vs C: Concorrência permite que múltiplas tarefas executem intercaladamente, compartilhando recursos; paralelismo permite que múltiplas tarefas executem simultaneamente, em vários núcleos para ganhar desempenho. Concorrência é fingir pro programa que ele tem acesso a vários recursos, enquanto na verdade tu tá só escalonando, enquanto paralelismo ele tem mesmo.

MT vs P vs D:

- aplicações multi-thread: usam várias threads dentro de um processo, melhor aproveitando a CPU; boa forma de organizar sistemas de software modernos
- aplicações paralelas: processos executando fisicamente ao mesmo tempo em diferentes processadores, para resolver um problema mais rapidamente ou maior
- aplicações distribuídas: rodam em máquinas/hosts diferentes, comunicando-se por rede, para distribuir trabalho, acessar dados distribuídos e computação paralela em larga-escala sobre a rede

Principais dificuldades da programação concorrente:

- Race Condition: quando threads acessam variáveis compartilhadas sem sincronização, gerando resultados incorretos
- Deadlock: quando threads ficam bloqueadas esperando por recursos uns dos outros
- Starvation: quando uma thread nunca é escalonada por falta de acesso a recursos

2) Explique o problema do produtor-consumidor e quais os requisitos pra não ter sobreposição de dados.

O problema do produtor-consumidor ocorre quando dois processos compartilham um buffer finito: o produtor insere itens e o consumidor os retira. Sem sincronização, pode haver

...inter e produtor: cada item é consumido e removido sem mensagens, pois há uma sobreposição de dados, com o produtor sobrescrevendo posições não consumidas (dados perdidos) ou o consumidor lendo posições vazias (dados inválidos). Para evitar isso, é necessário garantir:

- exclusão mútua: apenas um processo (produtor ou consumidor) pode acessar o buffer por vez, o que pode ser feito através de mutex ou semáforo binário
- sincronização por condição: produtor só insere se houver espaço livre e consumidor só retira se houver item disponível; isso pode ser feito com semáforos de contagem (full e empty) ou variáveis de condição (wait e signal)

2) Defina request-reply (isso é na verdade uma questão de marcar, mas não temos muito detalhe, a única coisa que sabemos sobre ela é que é sobre request-reply).

Request-reply é um padrão de comunicação síncrona no qual um processo cliente envia uma mensagem de request e outro processo (geralmente um servidor) processa e envia uma reply.

Algumas características principais dos protocolos request-reply são:

- geralmente é síncrono (cliente espera o reply) e confiável, mas pode ser assíncrono
- fornecem suporte direto pra RPC/RMI
- podem ser construídos sobre TCP ou UDP (pra UDP, precisa implementar coisas como ACKs, controles de fluxo, etc, mas sem overhead)
- a semântica pode ser at-most-once (CORBA), at-least-once (RPC) ou exactly-once (Java RMI)

Spector, 1982 definiu os protocolos Request (R), Request-Reply (RR) e Request-Reply-Ack (RRA)

- RR: implementa at-least-once e é possível fazer exactly-once através de um buffer pra saber quando a mensagem é duplicada (mas quando liberar o buffer?)
- RRA: mensagem de reply possui um ACK explícito, implementa at-least-once e é possível fazer exactly-once através de um buffer pra saber quando a mensagem é duplicada (mas quando liberar o buffer? quando receber o ACK)

3) Explique os algoritmos de Dekker, Peterson e Lamport (isso é na verdade uma questão de marcar, não temos muito detalhes, a única coisa que sabemos é que ela é sobre o algoritmo de Peterson).

Algoritmo de Dekker:

- projetado para dois processadores
- ideia: cada processo tem uma variável booleana compartilhada `querEntrar[i]` indicando se deseja entrar na seção crítica e uma variável compartilhada única `vez` que indica de quem é a vez de entrar e quem tem prioridade caso ambos queiram
- funcionamento:
 - o um processo `i` define `querEntrar[i] = true`
 - o se o outro também quiser entrar (ou seja, `querEntrar[j] = true`), `i` verifica `vez`
 - o se não for a sua vez, ele desiste momentaneamente (ou seja, `querEntrar[i] = false`) e espera
 - o quando chegar sua vez, ele entra na seção crítica
 - o ao sair, define `vez = j` e `querEntrar[i] = false`
- garante: exclusão mútua, progresso e espera limitada
- problema: difícil de generalizar para mais de dois processos e usa busy-wait
- fun fact: antes dessa versão final, várias outras foram feitas mas feriam alguns princípios como progresso, exclusão mútua, espera limitada e adiamento ilimitado

Algoritmo de Peterson:

- versão simplificada do Dekker, também para dois processos
- ideia: duas variáveis compartilhadas de novo, `querEntrar[i]` e `vez`
- funcionamento: o processo anuncia que quer entrar e cede a vez ao outro, mas só entra se o outro não quiser entrar
- pseudocódigo:

```
querEntrar[i] = true;
vez = j;
while (querEntrar[j] && vez == j)
    ; // espera (busy-wait)
--- seção crítica ---
querEntrar[i] = false;
```

- garante: exclusão mútua, progresso e espera limitada

- problema: difícil de generalizar para mais de dois processos e usa busy-wait

Algoritmo de Lamport:

- inspirado em uma padaria: cada processo pega um "número" (como uma senha), e entra na seção crítica em ordem crescente de número
- ideia:
 - o cada processo tem `escolhendo[i]`, indicando se está escolhendo seu número e `numero[i]` que indica o número de prioridade (ordem de chegada)
 - o o processo escolhe o número uma unidade maior que o maior atual
 - o entra na seção crítica apenas se nenhum outro processo está escolhendo número e todos os processos com número menor (ou número igual e ID menor, pra caso o mesmo número seja gerado pra dois processos) já saíram
- pseudocódigo:

```

escolhendo[i] = true;
numero[i] = 1 + max(numero[0..N-1]);
escolhendo[i] = false;

for (j = 0; j < N; j++) {
    while (escolhendo[j]) ; // espera o outro escolher
    while (numero[j] != 0 &&
           (numero[j] < numero[i] ||
            (numero[j] == numero[i] && j < i))) ;
}
--- seção crítica ---
numero[i] = 0;

```

- garante: exclusão mútua, progresso e espera limitada
- problema: implementação mais custosa e usa busy-wait

4) Explique o que é Remote Procedure Call e comente cada passo do processo (na verdade esse é um esboço de uma questão da prova do Weverton do semestre 2025/1, providenciada pelo aluno Matheus Oliveira Mews)

Remote Procedure Call (RPC) é um mecanismo que permite que um processo chame um procedimento em outro processo ou máquina como se fosse local. Envolve as seguintes etapas:

1. chamada local ao stub do cliente: o programa cliente chama uma função remota como se fosse local através de seu stub
2. empacotamento (marshalling): o stub converte os parâmetros em formato transmissível (bytes)
3. envio do request pela rede do servidor
4. recepção e desempacotamento (unmarshalling): o stub do servidor reconstrói os parâmetros e chama a função real
5. execução do procedimento real
6. empacotamento e envio da resposta: o resultado é convertido em mensagem de resposta e enviado pelo stub servidor
7. recebimento e desempacotamento: o stub do cliente reconstrói o resultado e o retorna ao programa

PRIMEIRA VERIFICAÇÃO 2024/1

1. (0.5pt) Considerando os conceitos abaixo, assinale as alternativas INCORRETAS:

- () Na coordenação da execução de ações entre processos, exclusão mútua refere-se a duas ou mais tarefas poderem estar simultaneamente em uma seção não crítica
- () Na coordenação da execução de ações entre processos, condição corresponde a atrasar a execução de uma tarefa até que uma determina condição seja verdadeira
- () Condição de corrida: diferentes processos acessam e manipulam dados diversos concorrentemente e o resultado da execução depende da ordem dos acessos
- () Seção crítica: trecho de código que manipula um recurso compartilhado e que não pode ser executado concorrentemente por outras tarefas

INCORRETA. Exclusão mútua trata de seções críticas, nas quais apenas um processo pode entrar por vez

CORRETA. Isso descreve o papel das variáveis de condição (wait e signal) usadas em monitores e